

UNIVERZITA PARDUBICE

Fakulta elektrotechniky a informatiky

Název projektu: **SportTracker – Software Design
Document**

Seminární práce z předmětu: **KIT/KRPW1 –
Ročníkový projekt I**

Program: **Webové technologie**

Jméno a příjmení: **Filip Krsek**

Kód studenta: **ST78966**

V

Praze

dne

26. 5. 2026

Obsah

1	Úvod	4
1.1	Účel dokumentu	4
1.2	Popis systému	4
1.3	Scope dokumentu	4
1.4	Použité technologie	5
1.5	Architektonický přístup	5
2	Architektura systému	6
2.1	Přehled architektury	6
2.2	Frontend vrstva	6
2.3	Backend vrstva	6
2.4	Databázová vrstva	7
2.5	Komunikace mezi vrstvami	7
2.6	Škálovatelnost systému	8
3	Databázový návrh	9
3.1	Přehled databázového návrhu	9
3.2	Hlavní entity systému	9
3.3	Relační vztahy	9
3.4	Rozšiřitelnost databázového modelu	10
3.5	ERD diagram	10
4	Backend návrh	12
4.1	Přehled backendové architektury	12
4.2	Controller vrstva	12
4.3	Repository vrstva	13
4.4	Entity vrstva	14
4.5	REST API komunikace	14
4.5.1	Players API	15
4.5.2	Trainings API	16
4.5.3	Attendance API	17
4.6	Budoucí rozšíření API	17
5	Frontend návrh	18
5.1	Přehled FE architektury	18
5.2	Struktura FE architektury	18

5.3	Dashboard.....	18
5.4	Trainings stránka	19
5.5	Players stránka.....	19
5.6	Statistics stránka	19
5.7	Profile stránka.....	19
5.8	Error handling.....	19
5.9	Komunikace FE a BE	20
5.10	Budoucí rozvoj FE části.....	20
6	Deployment diagram	21
7	Bezpečnost	22
	Závěr.....	23

1 Úvod

1.1 Účel dokumentu

Tento dokument představuje Software Design Document (SDD) projektu SportTracker. Dokument popisuje technický návrh systému, použitou architekturu, databázový model, komunikaci mezi jednotlivými vrstvami aplikace a návrh hlavních komponent systému.

Cílem dokumentu je definovat způsob implementace systému a popsat technická rozhodnutí použitá při návrhu a vývoji aplikace.

1.2 Popis systému

SportTracker je webová aplikace určená pro správu amatérských sportovních aktivit se zaměřením především na amatérský florbal. Systém umožňuje správu hráčů, organizaci tréninků, evidenci attendance a správu základních statistik.

Aplikace je navržena jako vícevrstvá webová aplikace využívající oddělený frontend, backend a databázovou vrstvu. Komunikace mezi frontendem a backendem probíhá pomocí REST API a JSON payloadů.

Současná implementace představuje MVP verzi systému s důrazem na jednoduchou rozšiřitelnost a možnost budoucí implementace dalších funkcionalit.

1.3 Scope dokumentu

Dokument popisuje:

- architekturu systému,
- návrh databáze,
- backendovou architekturu,
- frontendovou architekturu,
- REST API komunikaci,
- deployment návrh,
- bezpečnostní návrh,
- plánované budoucí rozšíření systému.

Součástí dokumentu nejsou detailní implementační detaily jednotlivých frontendových komponent ani detailní popis jednotlivých algoritmů systému.

1.4 Použité technologie

Frontendová část aplikace byla implementována pomocí technologií HTML, CSS a JavaScript. Frontend zajišťuje vykreslování uživatelského rozhraní, navigaci mezi jednotlivými stránkami a komunikaci s backendovou částí systému.

Backend systému je implementován pomocí programovacího jazyka Java a frameworku Spring Boot. Backend zajišťuje business logiku aplikace, zpracování REST API požadavků a komunikaci s databázovou vrstvou systému. Pro práci s databází je využíván framework Spring Data JPA společně s ORM nástrojem Hibernate.

Datová vrstva systému využívá relační databázi PostgreSQL, která slouží pro ukládání dat o hráčích, trénincích, attendance a dalších entitách systému.

Komunikace mezi frontendem a backendem probíhá pomocí REST API využívajícího JSON payloady.

Vrstva systému	Technologie
Frontend	HTML, CSS, JavaScript
Backend	Java Spring Boot
Databáze	PostgreSQL
ORM	Spring Data JPA / Hibernate
API komunikace	REST API + JSON

1.5 Architektonický přístup

Systém využívá layered architecture rozdělenou do tří hlavních vrstev:

- frontendová vrstva,
- backendová aplikační vrstva,
- databázová vrstva.

Frontend zajišťuje uživatelské rozhraní a komunikaci s backendem. Backend implementuje business logiku systému a zajišťuje komunikaci s databází. Databázová vrstva slouží pro ukládání a správu dat systému.

Architektura systému je navržena s důrazem na, jednoduchou rozšiřitelnost, oddělení jednotlivých vrstev systému, možnost budoucího škálování, a podporu budoucí implementace autentizace a autorizace uživatelů.

2 Architektura systému

2.1 Přehled architektury

Systém SportTracker je navržen jako třívrstvá webová aplikace využívající oddělenou frontendovou, backendovou a databázovou vrstvu. Jednotlivé vrstvy systému spolu komunikují pomocí REST API a JSON payloadů.

Architektura systému byla navržena s důrazem na:

- oddělení jednotlivých částí aplikace,
- jednoduchou rozšiřitelnost,
- možnost budoucího škálování,
- přehlednost implementace,
- podporu budoucí autentizace a autorizace uživatelů.

Současná implementace představuje monolitickou backendovou aplikaci využívající framework Spring Boot, která komunikuje s frontendovou částí systému a relační databází PostgreSQL.

2.2 Frontend vrstva

Frontendová vrstva systému zajišťuje:

- vykreslování uživatelského rozhraní,
- navigaci mezi jednotlivými stránkami,
- zobrazování dat,
- odesílání požadavků na backendové REST API.

Frontend je implementován pomocí:

- HTML,
- CSS,
- JavaScriptu.

Aplikace aktuálně využívá desktop-first přístup a obsahuje několik hlavních obrazovek:

- Dashboard,
- Trainings,
- Players,
- Statistics,
- Profile,
- Error handling page.

Frontend komunikuje s backendem pomocí HTTP requestů a JSON payloadů.

2.3 Backend vrstva

Backendová vrstva systému je implementována pomocí frameworku Spring Boot. Backend zajišťuje:

- business logiku aplikace,
- správu REST API endpointů,
- komunikaci s databází,
- práci s entitami systému.

Backend systému využívá layered architecture skládající se z:

- Controllers,
- Repositories,
- Entity vrstev.

Mezi hlavní entity systému patří:

- Player,
- Training,
- Attendance.

Backend poskytuje REST API endpointy pro:

- správu hráčů,
- správu tréninků,
- práci s attendance.

Komunikace probíhá pomocí JSON payloadů.

2.4 Databázová vrstva

Datová vrstva systému využívá relační databázi PostgreSQL. Databáze slouží pro ukládání:

- hráčů,
- tréninků,
- dat docházky,
- budoucích statistik a finančních dat.

Databázový model byl navržen s důrazem na:

- relační konzistenci,
- rozšiřitelnost,
- podporu budoucích funkcionalit,
- možnost rozšíření na více sportů.

Přístup k databázi je realizován pomocí:

- Spring Data JPA,
- Hibernate ORM frameworku.

2.5 Komunikace mezi vrstvami

Komunikace mezi frontendovou a backendovou vrstvou probíhá pomocí REST API přes HTTP protokol.

Frontend:

- odesílá HTTP requesty,
- přijímá JSON response,
- dynamicky vykresluje data.

Backend:

- zpracovává požadavky,
- komunikuje s databází,
- vrací JSON odpovědi frontendové vrstvě.

Pro podporu komunikace mezi odděleným frontendem a backendem je implementována CORS konfigurace.

2.6 Škálovatelnost systému

Architektura systému byla navržena s důrazem na možnost budoucího rozšiřování a škálování.

Systém podporuje:

- oddělené škálování frontendové a backendové vrstvy,
- rozšíření databázového modelu,
- podporu dalších sportů,
- implementaci autentizace a autorizace,
- rozšíření o další business moduly.

Databázový model i backendová architektura byly od začátku navrženy tak, aby bylo možné systém dále rozvíjet bez zásadních změn architektury aplikace.

3 Databázový návrh

3.1 Přehled databázového návrhu

Databázová vrstva systému SportTracker využívá relační databázi PostgreSQL. Současná implementace vychází z původního rozsáhlejšího databázového návrhu vytvořeného v prostředí Oracle SQL Developer Data Modeler, který je postupně převáděn do PostgreSQL databáze a upravován podle aktuálních potřeb implementace systému.

Databázový model byl navržen s důrazem na:

- relační konzistenci,
- přehlednost,
- jednoduchou rozšiřitelnost,
- podporu budoucích funkcionalit systému.

Aktuální MVP implementace obsahuje základní entity potřebné pro:

- správu hráčů,
- správu tréninků,
- evidenci attendance.

Původní návrh databáze však již počítá také s budoucím rozšířením systému o:

- finance,
- pokuty,
- zápasy,
- statistiky,
- správu více sportů,
- správu hal.

3.2 Hlavní entity systému

Současná implementace databáze obsahuje následující hlavní entity:

Player

Entita reprezentující hráče systému. Obsahuje základní informace o hráči, například jméno, příjmení, telefon nebo datum narození.

Training

Entita reprezentující jednotlivé tréninky systému. Obsahuje především datum tréninku a identifikátor tréninku.

Attendance

Entita sloužící pro evidenci attendance hráčů na konkrétních trénincích. Attendance propojuje entity Player a Training a eviduje plánovanou i skutečnou attendance hráče.

3.3 Relační vztahy

Databázový model využívá relační vazby mezi jednotlivými entitami.

Hlavní vztahy systému:

- jeden hráč může mít více attendance záznamů,
- jeden trénink může obsahovat více attendance záznamů,
- attendance propojuje hráče a konkrétní trénink.

Vztahy jsou implementovány pomocí JPA vazeb a foreign keys v PostgreSQL databázi.

3.4 Rozšiřitelnost databázového modelu

Databázový model byl od začátku navržen s důrazem na budoucí rozšiřitelnost systému.

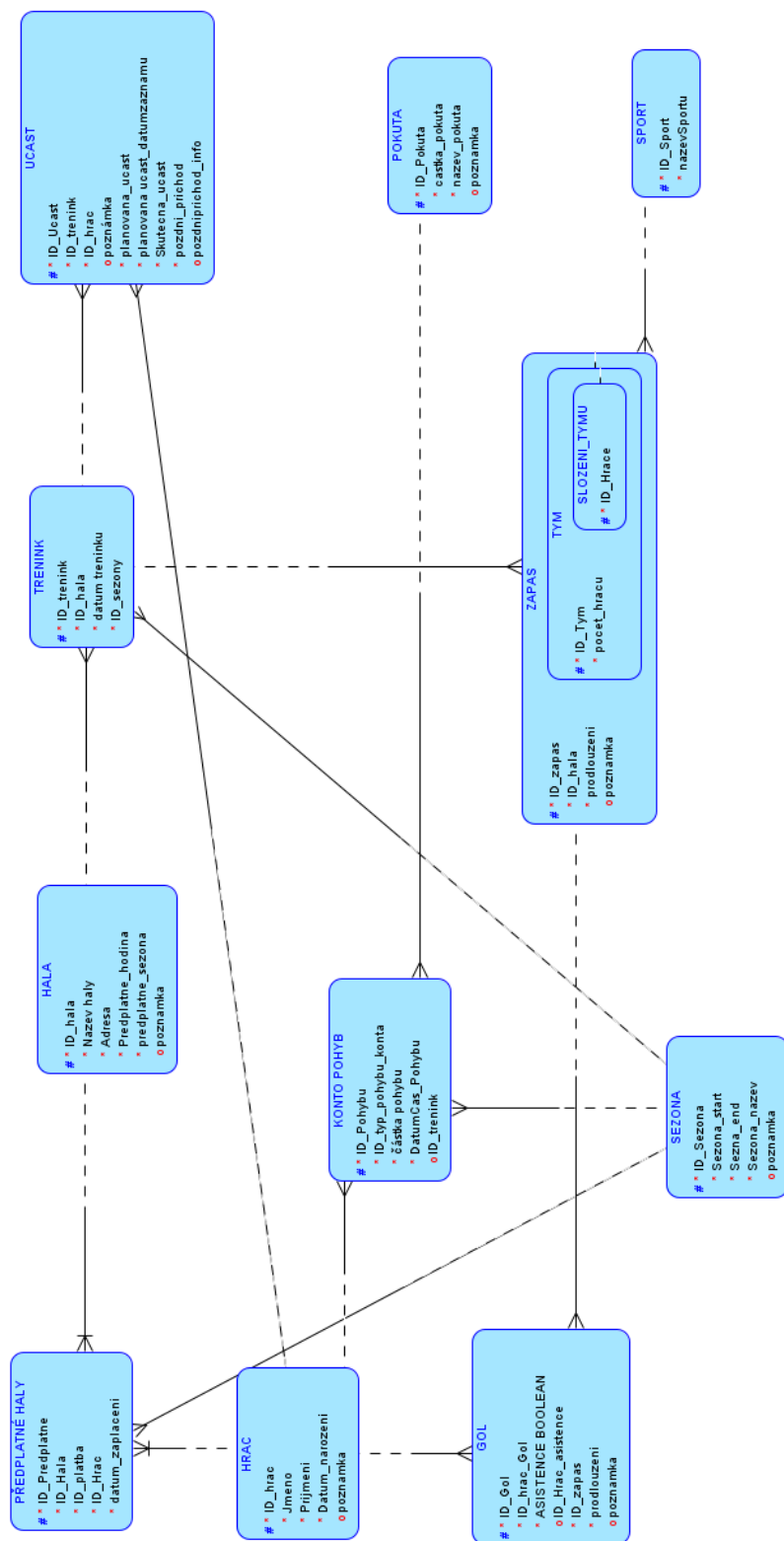
Původní návrh počítá například s budoucí implementací entit:

- Match,
- Team,
- Sport,
- Hall,
- Finance,
- Fine,
- UserRole.

Tento přístup umožňuje budoucí rozšíření systému bez zásadních změn databázové architektury.

3.5 ERD diagram

Níže uveden je ERD diagram zamýšlený k implementaci projektu vycházející z původního návrhu databáze pro Oracle.



4 Backend návrh

4.1 Přehled backendové architektury

Backend systému SportTracker je implementován pomocí frameworku Spring Boot v programovacím jazyce Java, verzi 17. Backend představuje aplikační vrstvu systému a zajišťuje:

- business logiku aplikace,
- správu REST API endpointů,
- komunikaci s databází,
- práci s entitami systému.

Architektura backendu využívá layered architecture rozdělenou do několika základních vrstev:

- Controller layer,
- Repository layer,
- Entity layer.

Backend systému je navržen jako REST API aplikace komunikující s frontendem pomocí HTTP requestů a JSON payloadů.

Hlavní vstupní bod backendové aplikace je implementován pomocí třídy SportTrackerApplication.

4.2 Controller vrstva

Controller vrstva zajišťuje:

- zpracování HTTP requestů,
- definici REST API endpointů,
- komunikaci mezi frontendem a backendem,
- vracení JSON response.

Současná implementace obsahuje následující controllery:

- PlayerController,
- TrainingController,
- AttendanceController.

PlayerController

Zajišťuje:

- získání seznamu hráčů,
- získání detailu hráče,
- editaci hráče.

Controller využívá endpointy:

- GET /players,
- GET /players/{id},
- PUT /players/{id}.

TrainingController

Zajišťuje:

- získání seznamu tréninků,
- získání detailu tréninku,
- vytvoření tréninku.

Controller využívá endpointy:

- GET /trainings,
- GET /trainings/{id},
- POST /trainings.

AttendanceController

Zajišťuje:

- získání attendance dat systému.

Controller využívá endpoint:

- GET /attendance.

4.3 Repository vrstva

Repository vrstva slouží pro komunikaci backendu s databází pomocí frameworku Spring Data JPA.

Současná implementace obsahuje:

- PlayerRepository,
- TrainingRepository,
- AttendanceRepository.

Repository vrstvy dědí funkcionalitu z JpaRepository a umožňují:

- CRUD operace,
- práci s databázovými entitami,
- automatickou správu základních databázových operací.

4.4 Entity vrstva

Entity vrstva reprezentuje databázové entity systému pomocí JPA modelů.

Současná implementace obsahuje entity:

- Player,
- Training,
- Attendance.

Entity využívají:

- JPA anotace,
- relační vazby,
- ORM mapování pomocí Hibernate.

Player

Reprezentuje hráče systému a obsahuje základní profilové informace.

Training

Reprezentuje jednotlivé tréninky systému.

Attendance

Propojuje hráče a tréninky a eviduje attendance informace

4.5 REST API komunikace

Backend systému SportTracker poskytuje REST API určené pro komunikaci mezi frontendovou a backendovou vrstvou aplikace. API využívá HTTP requesty a JSON payloady.

Architektura API vychází z principů REST (Representational State Transfer):

- jednotlivé entity systému jsou reprezentovány pomocí endpointů,
- komunikace probíhá pomocí standardních HTTP metod,
- data jsou přenášena ve formátu JSON,
- endpointy jsou navrženy s důrazem na jednoduchost a rozšiřitelnost.

4.5.1 Players API

1) GET /players

Vrací seznam všech hráčů v systému.

Payload: Request payload není vyžadován

Response: JSON

```
[
  {
    "id": 1,
    "firstName": "Filip",
    "lastName": "Novák",
    "phone": "+420777123456",
    "birthDate": "2000-05-12",
    "note": "Kapitán týmu"
  }
]
```

2) GET /players/{id}

Vrací detail jednoho hráče

Payload: Request payload není vyžadován

Response: JSON

```
{
  "id": 1,
  "firstName": "Filip",
  "lastName": "Novák",
  "phone": "+420777123456",
  "birthDate": "2000-05-12",
  "note": "Kapitán týmu"
}
```

3) PUT /players/{id}

Editace hráče

Payload: JSON

```
{
  "firstName": "Filip",
  "lastName": "Novák",
  "phone": "+420777123456",
  "birthDate": "2000-05-12",
  "note": "Updated profile"
}
```

Response: JSON

```
{
  "id": 1,
  "firstName": "Filip",
  "lastName": "Novák",
  "phone": "+420777123456",
  "birthDate": "2000-05-12",
  "note": "Updated profile"
}
```

4.5.2 Trainings API

1) GET /trainings

Získání seznamu tréninků

Payload: Request payload není vyžadován

Response: JSON

```
[
  {
    "id": 1,
    "trainingDate": "2026-06-01"
  }
]
```

2) GET /trainings/{id}

Získání detailu jednoho tréninku

Payload: Request payload není vyžadován

Response: JSON

```
[
  {
    "id": 1,
    "trainingDate": "2026-06-01"
  }
]
```


- 3) POST /trainings
Vytvoření tréninku
Payload: JSON

```
{  
  "trainingDate": "2026-06-01"  
}
```

Response: JSON

```
{  
  "id": 7,  
  "trainingDate": "2026-06-01"  
}
```

4.5.3 Attendance API

- 1) GET /attendance
Získání attendance dat
Payload: Request payload není vyžadován.
Response: JSON

```
[  
  {  
    "id": 1,  
    "plannedAttendance": true,  
    "actualAttendance": true  
  }  
]
```

4.6 Budoucí rozšíření API

Současná implementace REST API představuje základní MVP verzi backendové komunikace systému. Do budoucna je plánováno rozšíření API například o:

- autentizaci uživatelů,
- JWT token-based authorization,
- správu financí,
- správu pokut,
- správu statistik,
- správu zápasů,
- podporu více sportů,
- notifikační endpointy.

Architektura API byla navržena s důrazem na jednoduchou rozšiřitelnost a možnost budoucího rozvoje systému bez zásadních změn existujících endpointů.

5 Frontend návrh

5.1 Přehled FE architektury

Frontendová část systému SportTracker představuje uživatelské rozhraní aplikace a zajišťuje komunikaci uživatele s backendovou částí systému. Frontend je implementován pomocí technologií HTML, CSS a JavaScript.

Architektura frontendové části byla navržena s důrazem na:

- jednoduchost implementace,
- přehlednost uživatelského rozhraní,
- oddělení jednotlivých obrazovek aplikace,
- jednoduchou komunikaci s REST API backendu,
- možnost budoucího rozšíření systému.

Současná implementace představuje desktop-first MVP verzi aplikace.

5.2 Struktura FE architektury

Frontend aplikace je rozdělen do několika hlavních obrazovek reprezentujících jednotlivé části systému.

Mezi hlavní stránky systému patří:

- Dashboard,
- Trainings,
- Players,
- Statistics,
- Profile,
- Error handling page.

Jednotlivé stránky jsou propojeny pomocí sidebar navigace.

Frontend využívá komponentový přístup založený na:

- opakovaně použitelných HTML sekcích,
- sdíleném CSS stylování,
- JavaScript logice pro načítání dat z backendu.

5.3 Dashboard

Dashboard představuje hlavní vstupní obrazovku systému. Stránka slouží pro:

- zobrazení nadcházejících tréninků,
- rychlý přístup k hlavním funkcionalitám systému,
- základní přehled o aktivitách aplikace.

Dashboard obsahuje:

- navigační prvky,
- seznam tréninků,
- odkazy na další části systému.

5.4 Trainings stránka

Stránka Players slouží pro správu a zobrazení hráčů systému.

Stránka umožňuje:

- zobrazení seznamu hráčů,
- zobrazení detailu hráče,
- editaci profilových informací hráče.

Komunikace s backendem probíhá pomocí REST API endpointů Players API.

5.5 Players stránka

Stránka Players slouží pro správu a zobrazení hráčů systému.

Stránka umožňuje:

- zobrazení seznamu hráčů,
- zobrazení detailu hráče,
- editaci profilových informací hráče.

Komunikace s backendem probíhá pomocí REST API endpointů Players API.

5.6 Statistics stránka

Stránka Statistics slouží pro zobrazení statistik hráčů a tréninků.

Současná implementace obsahuje:

- demonstrační statistická data,
- filtrování statistik podle sportu,
- základní přehled statistik systému.

Některé části statistik aktuálně využívají mock data z důvodu postupné implementace backendové logiky.

5.7 Profile stránka

Profile stránka umožňuje zobrazení a editaci základních profilových informací hráče.

Stránka umožňuje:

- editaci jména,
- editaci příjmení,
- editaci telefonního čísla,
- editaci poznámky,
- aktualizaci dat pomocí REST API komunikace.

Změny profilových dat jsou ukládány do PostgreSQL databáze prostřednictvím backendové vrstvy systému.

5.8 Error handling

Frontend aplikace obsahuje základní error handling mechanismus sloužící pro přesměrování uživatele při přístupu na neimplementované nebo neexistující části systému.

Error handling page slouží především pro:

- zachycení neexistujících navigačních cest,
- informování uživatele o neimplementovaných funkcionalitách,
- zajištění konzistence navigace aplikace.

5.9 Komunikace FE a BE

Frontend komunikuje s backendovou částí systému pomocí:

- HTTP requestů,
- REST API endpointů,
- JSON payloadů.

Komunikace je realizována pomocí JavaScript fetch API.

Frontend:

- odesílá requesty na backend,
- přijímá JSON response,
- dynamicky aktualizuje obsah jednotlivých stránek aplikace.

5.10 Budoucí rozvoj FE části

Do budoucna je plánováno rozšíření frontendové části systému například o:

- responzivní mobilní rozhraní,
- autentizaci uživatelů,
- role-based zobrazení obsahu,
- pokročilé statistiky,
- modernější layout
- notifikační systém,

6 Deployment diagram

Systém SportTracker je navržen jako třívrstvá webová aplikace rozdělená na:

- frontendovou vrstvu,
- backendovou vrstvu,
- databázovou vrstvu.

Frontend představuje klientskou část aplikace běžící ve webovém prohlížeči uživatele.

Frontend je implementován pomocí HTML, CSS a JavaScriptu a komunikuje s backendem pomocí REST API a HTTP requestů.

Backend systému je implementován pomocí frameworku Spring Boot. Backend zajišťuje business logiku aplikace, správu REST API endpointů a komunikaci s databází PostgreSQL.

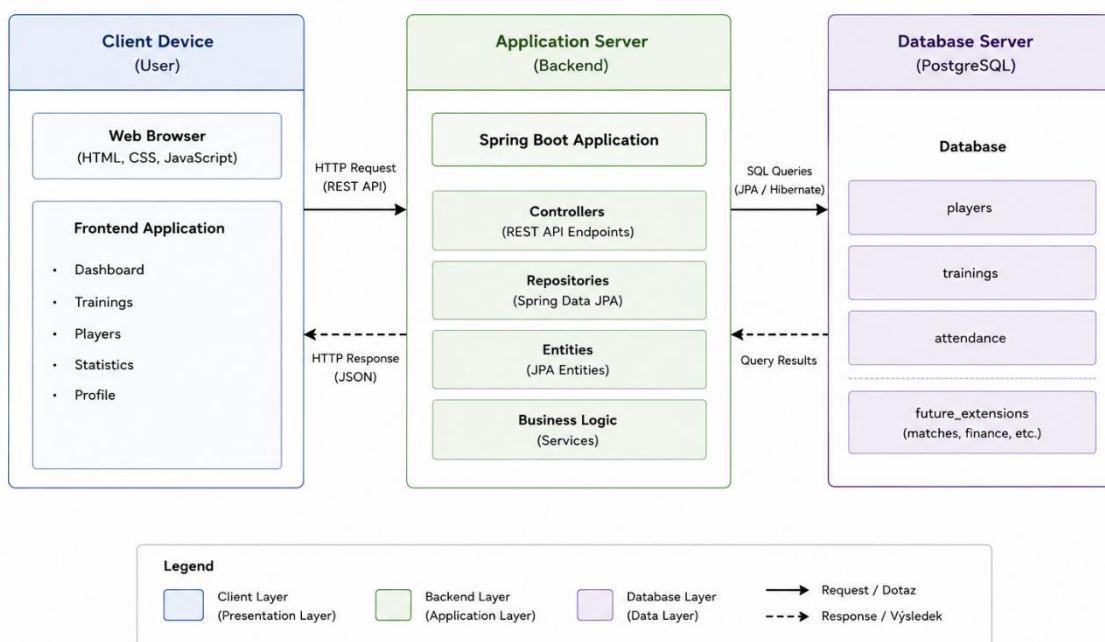
Databázová vrstva využívá relační databázi PostgreSQL sloužící pro ukládání dat o hráčích, trénincích a attendance.

Komunikace systému probíhá následujícím způsobem:

1. Uživatel komunikuje s frontendovou částí aplikace pomocí webového prohlížeče.
2. Frontend odesílá HTTP requesty na backendové REST API.
3. Backend zpracovává požadavky a komunikuje s databází.
4. Backend vrací JSON response frontendové vrstvě.
5. Frontend aktualizuje uživatelské rozhraní.

Architektura systému byla navržena s důrazem na jednoduchou rozšiřitelnost a možnost budoucího odděleného škálování frontendové a backendové vrstvy.

Deployment Diagram - SportTracker



7 Bezpečnost

Současná verze systému SportTracker představuje MVP studentského projektu a neobsahuje implementovanou autentizaci ani autorizaci uživatelů. Architektura systému však byla od začátku navržena s důrazem na možnost budoucí implementace bezpečnostních mechanismů.

Komunikace mezi frontendem a backendem probíhá pomocí REST API a JSON payloadů. Backend systému je připraven na budoucí implementaci:

- JWT autentizace,
- role-based access control,
- HTTPS komunikace,
- oddělení uživatelských rolí.

Do budoucna je plánováno rozdělení oprávnění mezi role:

- Admin,
- Organizátor,
- Hráč.

Současná implementace zároveň obsahuje základní error handling pro neplatné požadavky a neexistující stránky systému.

Závěr

Dokument Software Design Document popisuje technický návrh systému SportTracker, jeho architekturu, databázový model, frontendovou a backendovou část aplikace i způsob komunikace mezi jednotlivými vrstvami systému.

Výsledná architektura byla navržena s důrazem na jednoduchost, přehlednost a možnost budoucího rozšíření systému. Současná implementace představuje funkční MVP verzi aplikace připravenou pro další rozvoj a implementaci pokročilejších funkcionalit.